Лекция 3. Структуры данных. Перегрузка функций. Вектор. Строка

1. Структура `struct`
   - Размер структуры
   - Создание объекта
   - Инициализация полей значениями по умолчанию
   - Доступ к полям структуры. Оператор `.`
   - Доступ к полям структуры. Оператор `->`
2. Объединение `union`
   - Использование неактивной переменной
   - Анонимные объединения
3. Пара `std::pair`
   - Инициализация и доступ к полям
   - Доступ к полям с помощью `std::get` *(C++11)*
   - Создание с помощью `std::make_pair`
4. Кортеж `std::tuple` *(C++11)*
   - Инициализация
   - Доступ к полям с помощью `std::get`
   - Создание с помощью `std::make_tuple`
5. Функция связывания `std::tie` *(C++11)*
6. Структурное связывание (*structured bindings*) *C++17*
7. Перегрузка функций
   - Перегрузка операторов
8. Асимптотическая сложность
9. Последовательные контейнеры
10. Контейнер `std::vector`
    - Инициализация `std::vector`
    - Устройство `std::vector`
    - Обзор методов `std::vector`
    - Вместимость контейнера `std::vector`
    - Добавление в конец `std::vector`
    - Доступ к элементам `std::vector`
    - Итераторы
    - Итераторы `std::vector`
    - Итерирование `std::vector`
    - Метод `resize()`
    - Метод `reserve()`
    - Метод `pop_back()`
    - Метод `clear()`
    - Остальные методы `std::vector`

- Инвалидация итераторов `std::vector`

11. Строка `std::string`
    - Оптимизация коротких строк (*SSO*)
    - Создание строки
    - Литерал `""s` (*C++14*)
    - Общие с `std::vector` методы
    - Конкатенация и присваивающие операторы
    - Изменение строки
    - Поиск в строке
    - Сравнение строк
    - Преобразование чисел и строк (*C++11*)
    - Ввод строки

Структура `struct`

Структура представляет собой пользовательский тип данных, позволяющий объединять несколько переменных разных типов под одним именем.

Синтаксис определения: `struct <name> { <variables>... };`

- `<name>` - имя структуры, является именем нового пользовательского типа данных
- `<variables>...` - ноль и более переменных

Синтаксис объявления: `struct <name>;`

- может пригодиться, когда необходимо использовать структуру раньше, чем она определена.

Переменная, объявленная внутри структуры, называется **полем** структуры.

Гарантируется, что поля структуры располагаются в памяти в порядке объявления полей внутри структуры.

Размер структуры

Размер структуры зависит от размера входящих в неё полей и выравнивания (*alignment*).

```
struct Point { int x; int y; };
struct IID { int x; int y; double d; };
struct IDI { int x; double d; int y; };
struct IDIB { int x; double d; int y; bool b; };

std::cout << sizeof(Point) << std::endl; // 8
std::cout << sizeof(IID) << std::endl;   // 16
```

```
std::cout << sizeof(IDI) << std::endl;   // 24
std::cout << sizeof(IDIB) << std::endl;  // 24
```

Создание объекта

Имя структуры используется в качестве имени типа при создании объекта.

Синтаксис: `<struct_name> <var_name>;`

Рекомендуется сразу инициализировать поля объекта, сделать это можно, используя фигурные скобки `{}` и перечисляя значение полей в том порядке, в каком они определены в структуре.

```
struct Product {
    int count;
    double price;
};

Product obj;
Product obj_emp{};
Product obj_empty = {};
Product obj_init = {10, 300.0};
Product obj_part = {10};        // partial
Product obj_agr{10, 300.0};     // uniform
```

- в случае `obj` поля неинициализированы, чтение приведет к **UB**
- частичная инициализация приведет к предупреждению `-Wmissing-field-initializers`, которое при включенной опции `-Werror` становится ошибкой

Также, начиная с C++20 можно использовать `()` непосредственно после имени объекта. **НО** в случае пустых аргументов, это будет считываться как объявление функции (`Product obj();`). Поэтому рекомендуется использовать фигурные скобки `{}` для инициализации.

Имя типа структуры может совпадать с каким-либо объектом, тогда при объявлении можно разрешить конфликт имен добавив `struct` перед именем типа структуры (как правило, лучше **ИЗБЕЖАТЬ** совпадающих названий):

Синтаксис: `struct <struct_name> <var_name>;`

```
struct Product {
    int count;
    double price;
};

std::string Product = "some_product_name";
// Product p = {10, 300.0};     // compile error
struct Product p = {10, 300.0}; // OK
```

Можно объявить структуру сразу с созданием объектов данного типа, **НО** лучше так **НЕ** делать, особенно в глобальной области видимости. Иногда применяется, когда структура находится внутри класса (другой структуры) и необходимо сразу создать поле данного типа.

Синтаксис: `struct <struct_name> { <variables>... } <var_name1>, <var_name2>...;`

```
struct Point {
    int x;
    int y;
} p1, p2;
```

Инициализация полей значениями по умолчанию

Начиная с *C++11* можно инициализировать поля значениями по умолчанию через `=` при определении структуры. Тогда при создании объекта данного типа поля будут инициализированы значениями по умолчанию.

```
struct Product {
    int count = 0;
    double price = 100.0;
};

Product one;
Product two = {};
Product three{};
```

Доступ к полям структуры. Оператор `.`

Доступ к полям структуры через объект типа структуры осуществляется через оператор `.` точка. После имени объекта ставится `.` и затем имя поля:

```
struct Product {
    int count;
    double price;
};

Product obj;
obj.count = 10;
obj.price = 300.0;
```

Доступ к полям структуры. Оператор `->`

Доступ к полям структуры через указатель на объект типа структуры осуществляется через оператор `->` стрелочка. После указателя на объект ставится `->` и затем имя поля:

```
struct Product {
    int count;
    double price;
};

Product obj;
Product* ptr = &obj;
ptr->count = 10;
ptr->price = 300.0;
```

Объединение `union`

Объединение представляет собой пользовательский тип данных, который позволяет хранить разные типы данных в одной области памяти. В один момент может храниться только один тип данных. Размер объединения соответствует наибольшему по размеру типу, входящего в объединение.

Синтаксис: `union <union_name> { <variables>... };`

```
union Union {
    char c;
    int i;
    double d;
};
```

Все входящие в объединение типы переменных расположены по одному адресу.

Доступ к переменным аналогичен структурам.

Использование неактивной переменной

Активная переменная определяется последним присваиванием, при этом время жизни предыдущей активной переменной заканчивается. Использование неактивной переменной это **UB**. Однако, современные компиляторы, как правило, предоставляют расширение языка и позволяют читать неактивные переменные. Таким образом, можно получить доступ к отдельным байтам.

```
union Color {
    uint32_t value;
    struct {
        uint8_t a;
        uint8_t b;
        uint8_t g;
```

```
        uint8_t r;
    } components;
};

Color c;
c.value = 0x00FF9955;

std::cout << "Red: " << +c.components.r << std::endl;     // 0
std::cout << "Green: " << +c.components.g << std::endl;   // 255
std::cout << "Blue: " << +c.components.b << std::endl;    // 153
std::cout << "Alpha: " << +c.components.a << std::endl;   // 85
```

- но это по-прежнему **UB** и платформо-зависимый код, на *little-endian* и *big-endian* различный вывод.

Аналогично можно работать отдельно с битами:

```
union HardwareRegister {
    uint32_t raw;
    struct {
        uint32_t enable : 1;
        uint32_t mode : 3;
        uint32_t clock_divider : 4;
        uint32_t reserved : 24;
    } bits; // 1 + 3 + 4 + 24 = 32
};

HardwareRegister reg;
reg.raw = 0;
reg.bits.enable = 1;
reg.bits.mode = 5;
reg.bits.clock_divider = 8;
```

Анонимные объединения

Можно создавать анонимные объединения, тогда все внутренние имена переменных попадают во внешнюю область видимости объединения и не должно быть конфликта имен.

```
struct Variant {
    enum Type { INT, FLOAT, STRING } type;

    union {
        int intValue;
        float floatValue;
        char stringValue[50];
    };
};
```

Пара `std::pair`

Пара `std::pair` - это шаблонная структура, позволяющая хранить переменные двух типов в полях `first` и `second`.

Заголовочный файл: `<utility>`

Объявление: `std::pair<T1, T2> <name>;`

- `T1` - тип первой переменной
- `T2` - тип второй переменной

Методы:

- `swap` - для обмена значений с другой парой

Доступны операторы сравнения, сравнение производится лексикографически.

Пара используется в стандартных ассоциативных контейнерах, при возврате ключа и значения контейнера. А также, может быть использована при возврате двух значений из функции. Как правило, это менее информативно, чем структура с понятными именами полей.

Инициализация и доступ к полям

Доступ к соответствующим переменным в паре, осуществляется через имена полей `first` и `second`.

Инициализировать пару:

```
std::pair<char, int> p;                // initialized by default value
p.first = 'c';
p.second = static_cast<int>(p.first);

std::pair<bool, std::string> p1 = {true, "Book"};
std::pair<int, double> p2(10, 3.14);    // ctor
std::pair<bool, double> p3{true, 2.73}; // uniform C++11
```

Доступ к полям с помощью `std::get` (*C++11*)

В заголовочном файле `<utility>` есть шаблонная функция `std::get`, которая на вход принимает пару и возвращает ссылку на поле пары.

Может работать по индексу:

- `0` - `first`
- `1` - `second`
- другие значения приведут к ошибке компиляции

Начиная с C++14, может работать по типу переменной, в случае одинаковых типов будет ошибка компиляции.

```
std::pair<bool, int> p = {true, 10};
std::cout << '(' << std::get<0>(p) << ", " << std::get<1>(p) << ')' <<
std::endl;
std::cout << '(' << std::get<bool>(p) << ", " << std::get<int>(p) << ')' <<
std::endl;
std::get<bool>(p) = false;
std::get<1>(p) = 0;
```

Создание с помощью `std::make_pair`

В заголовочном файле `<utility>` есть шаблонная функция `std::make_pair` принимающая два аргумента и возвращающая пару из двух аргументов. Удобство заключается в возможности использования вывода типа переменной, если написать ключевое слово `auto`.

В заголовочном файле `<functional>` есть шаблонная функция `std::ref`, позволяющая получить обертку над ссылкой на объект, которая при использовании разворачивается непосредственно в ссылку на объект.

```
auto p1 = std::make_pair(15, 'c');    // std::pair<int, char>
auto p2 = std::make_pair(true, 3.14); // std::pair<bool, double>

int value = 3;
int array[] = {1, 2, 3};
auto p3 = std::make_pair(std::ref(value), array); // std::pair<int&, int*>
```

Кортеж `std::tuple` (*C++11*)

Кортеж `std::tuple` - шаблонный класс, позволяющий хранить произвольное число аргументов различных типов.

Заголовочный файл: `<tuple>`

Объявление: `std::tuple<T...> <name>;`

- `T...` - произвольное количество различных типов

Методы:

- `swap` - для обмена значений с другим аналогичным по типам кортежем

Для кортежа определены операторы сравнения.

Инициализация

Инициализировать кортеж можно привычными способами:

```
std::tuple<char, int> t;                                    // default ctor
std::tuple<bool, std::string, int> t1 = {true, "Book", 10};  // copy
std::tuple<int, double, int> t2(10, 3.14, -1);               // ctor
std::tuple<bool, double, int, char> t3{true, 2.73, 0, '0'};  // uniform C++11
```

Доступ к полям с помощью `std::get`

Поскольку кортеж не имеет прямого доступа к полям, то получить значения полей кортежа или изменить их можно с помощью шаблонной функции `std::get`.

Функция принимает кортеж, а в качестве параметров шаблона внутри `<>` указывается либо порядковый номер начиная с `0` и меньше количества переменных, либо тип данных (C++14), если он уникален для кортежа

```
std::tuple<float, char, int> tpl(10., 'a', 10);
std::get<0>(tpl) += 1;
std::get<1>(tpl) += 1;
std::get<2>(tpl) += 1;
// tpl = {11, 'b', 11};
std::get<float>(tpl) += 1;
std::get<char>(tpl) += 1;
std::get<int>(tpl) += 1;
// tpl = {12, 'c', 12};
```

Создание с помощью `std::make_tuple`

В заголовочном файле `<tuple>` есть шаблонная функция `std::make_tuple` принимающая произвольное число аргументов различных типов и возвращающая кортеж из переданных аргументов.

Удобство заключается в возможности использования вывода типа переменной, если написать ключевое слово `auto`.

```
auto t1 = std::make_tuple(15, 'c', 3.14); // std::tuple<int, char, double>

struct Product { int count; double price; };
auto t2 = std::make_tuple(true, Product{10, 3.14}); // std::tuple<bool, Product>
std::cout << std::get<1>(t2).count << " " << std::get<1>(t2).price;
```

Функция связывания `std::tie` (*C++11*)

Шаблонная функция, позволяющая создать кортеж ссылок на переданные аргументы или ссылку на специальный объект `std::ignore`

- применяется для лексикографического сравнения структур:

```
struct Pair { int value; std::string name; } p1, p2;
std::tie(p1.value, p1.name) < std::tie(p2.value, p2.name);
```

- для распаковки кортежа или пары (так как пара конвертируется в кортеж):

```
int i;
double d;
std::tie(i, std::ignore, d) = std::tuple(1, "Test", 2.0);
```

Структурное связывание (*structured bindings*) *C++17*

Structured bindings - синтаксическая конструкция, введенная в *C++17*, которая позволяет распаковывать составляющие составных типов данных в отдельные переменные.

Синтаксис: `<cv> auto <ref> [<name>...] = <var>;`

Работает с массивами, структурами, парами, кортежами.

```
int array[] = {1, 2};

auto& [first, second] = array;
first = 3;
second = 4; // array = {3, 4};
```

```
struct Product { int count; double price; };

Product prod{10, 100.0};
auto [count, price] = prod;
auto& [ref_count, ref_price] = prod;
const auto& [cref_count, cref_price] = prod;
```

```
std::pair<bool, int> p(true, 10);
const auto& [b, i] = p;
```

```
std::tuple<float, char, int> tpl(10., 'a', 10);
auto [f, c, i] = tpl;
```

Перегрузка функций

Перегрузка функций позволяет иметь функции с одинаковыми именами при различных входных аргументах.

Перегрузить функцию можно только по аргументам, изменение возвращаемого типа без изменения аргументов **НЕ** является перегрузкой.

CV-квалификаторы изменяют тип, поэтому добавление константности к типу, на который указывает указатель или ссылается ссылка (`const int*`, `const int&`), является перегрузкой. Верхнеуровневый `const` (`int* const`, `const int`) отбрасывается и перегрузкой **НЕ** является.

```
void PrintValues(const int* begin, const int* end) {}
void PrintValues(int* begin, int* end) {}
void PrintValues(const int* array, size_t size) {}
void PrintValues(const Point& point) {}
// Point PrintValues(const Point& point) {} // compile error
```

Функция, имеющая значения по умолчанию, может быть вызвана с меньшим количеством аргументов.

```
void PrintValues(int x, float f = 0.0, char c = 'a') {
    std::cout << "x = " << x << ", f = " << f << ", c = " << c << std::endl;
}

PrintValues(x, f, c);
PrintValues(x, f);
PrintValues(x);
```

Перегрузка операторов

Операторы также можно перегрузить. Перегруженный оператор представляет собой функцию. Перегрузка позволяет использовать операторы для своих типов данных.

Синтаксис: `<return_type> operator<op_symbol>(<arguments>...);`

- `<op_symbol>` - символьное обозначение оператора (`+`, `<<`, `==`, ...)
- `<arguments>...` - один или два аргумента (для унарного и бинарного операторов)
- `<return_type>` - тип возвращаемого значения

НЕЛЬЗЯ перегрузить операторы: `::`, `?:`, `.`, `.*`, `typeid`, `sizeof`, `alignof`

Перегрузка оператора `<<` для вывода в поток:

```
struct Pair {
    int value;
    std::string name;
```

```
};

std::ostream& operator<<(std::ostream& os, const Pair& pair) {
    os << '(' << pair.value << ", " << pair.name << ')';
    return os;
}
```

Перегрузка операторов сравнения:

```
bool operator==(const Pair& lhs, const Pair& rhs) {
    return lhs.value == rhs.value && lhs.name == rhs.name;
}

bool operator<(const Pair& lhs, const Pair& rhs) {
    if (lhs.value == rhs.value) {
        return lhs.name < rhs.name;
    }
    return lhs.value < rhs.value;
//    return (lhs.value == rhs.value) ? lhs.name < rhs.name : lhs.value <
rhs.value;
//    return std::tie(lhs.value, lhs.name) < std::tie(rhs.value, rhs.name);
}

bool operator!=(const Pair& lhs, const Pair& rhs) {
    return !(lhs == rhs);
}

bool operator>(const Pair& lhs, const Pair& rhs) {
    return rhs < lhs;
    // return !(lhs < rhs || lhs == rhs);
}

bool operator>=(const Pair& lhs, const Pair& rhs) {
    return !(lhs < rhs);
}

bool operator<=(const Pair& lhs, const Pair& rhs) {
    return lhs < rhs || lhs == rhs;
}
```

Асимптотическая сложность

Асимптотическая сложность — это математический способ оценки, как время выполнения или объём потребляемой памяти алгоритма растут с увеличением размера входных данных (`n`), когда `n` стремится к бесконечности.

При оценке игнорируются константы и младшие члены в формуле, что позволяет сосредоточиться на основном факторе масштабирования. Оценка позволяет сравнивать эффективность алгоритмов независимо от аппаратного обеспечения.

Обычно используется O-нотация («О большое»), которая задаёт верхнюю оценку роста функции. Как правило, оценивают наихудший случай, но можно оценивать и средний.

Распространенная асимптотическая сложность:

1. $O(1)$ — Константная сложность
   - Время выполнения не зависит от размера входных данных.
2. $O(log(n))$ — Логарифмическая сложность
   - Медленный рост. Удвоение `n` увеличивает время работы лишь на константу.
3. $O(n)$ — Линейная сложность
   - Время растет прямо пропорционально n.
4. $O(n \cdot log(n))$ — Линейно-логарифмическая сложность
   - Золотой стандарт для эффективных алгоритмов сортировки общего назначения.
5. $O(n^a)$ — Степенная сложность
   - Время растет пропорционально степени размера данных. При больших `n` становится очень медленно.
6. $O(2^n)$ — Экспоненциальная сложность
   - Чрезвычайно быстрый рост. Практически неприменимо для задач с `n` > 30-40.
7. $O(n!)$ — Факториальная сложность
   - Рост быстрее экспоненциальной. Характерна для задач полного перебора всех перестановок.

Амортизированная сложность — средняя стоимость операции в длинной последовательности операций. Отдельный `push_back` с реаллокацией стоит `O(n)`, но реаллокации происходят всё реже, и в среднем на одну операцию приходится `O(1)`

Последовательные контейнеры

Стандартная библиотека *STL* содержит множество шаблонных классов, которые представляют различные структуры данных. Как правило, их называют контейнерами. Контейнеры способны хранить произвольный тип данных. Контейнеры представляют собой коллекцию элементов определенного типа. Контейнеры предоставляют удобный способ взаимодействовать с элементами

В качестве параметров шаблона внутри угловых скобок `<>` обычно необходимо указать тип данных, который использует контейнер.

У многих последовательных контейнеров определен оператор доступа по индексу `[]`

Контейнер `std::vector`

Контейнер `std::vector` предназначен для хранения массива данных определенного типа непрерывно в динамической памяти.

Контейнер обеспечивает быстрое добавление элементов в конец и увеличение размера блока памяти под элементы при необходимости.

Для гарантии непрерывности памяти, при добавлении элемента в заполненный контейнер, происходит **реаллокация** всех элементов контейнера.

Заголовочный файл: `<vector>`

Синтаксис: `std::vector<T> <name>;`

- `T` - произвольный тип данных
- `<name>` - имя переменной

Инициализация `std::vector`

Доступны множество способов инициализации вектора:

```
std::vector<int> v1;                // empty
std::vector<int> v2(v1);            // copy ctor from other vector
std::vector<int> v3 = v1;           // copy ctor from other vector
// ctor
std::vector<int> v4(5);             // 5 elements with zero value
std::vector<int> v5(5, 2);          // 5 elements equal 2
// initializer list
std::vector<int> v6{1, 2, 4, 5};    // with elements 1, 2, 4, 5
std::vector<int> v7 = {1, 2, 3, 5}; // with elements 1, 2, 3, 5
// iterators
std::vector<int> v8(v7.begin(), v7.end()); // same as v7
```

Устройство `std::vector`

Контейнер должен иметь **указатель на данные** (адрес начала памяти в куче, выделенной под хранение элементов вектора), **вместимость выделенной памяти** (сколько элементов данного типа можно положить), **текущее количество элементов** (сколько элементов данного типа уже положено в контейнер).

Условно можно представить себе, что хранится 3 поля: указатель на данные, размер данных *size*, размер выделенной памяти *capacity*

Контейнер имеет размер 24 байта для 64-битных libstdc++/libc++. Стандарт размера не задает.

Фактически, стандартный контейнер устроен несколько иначе, внутри хранится 3 указателя: на начало данных, за конец выделенной памяти, за конец элементов

Элементы контейнера располагаются не на **стеке**, а в области памяти, называемой куча.

Обзор методов `std::vector`

Контейнер имеет встроенные функции для работы с его содержимым такие функции называются **методы**.

Доступ к элементам:

- `at(size_t pos)` - метод, принимающий индекс элемента и возвращающий ссылку
- `[size_t pos]` - оператор `[]`, принимающий индекс и возвращающий ссылку
- `front()` - метод, возвращающий ссылку на первый элемент
- `back()` - метод, возвращающий ссылку на последний элемент
- `data()` - метод, возвращающий указатель на начало данных

Вместимость контейнера:

- `empty()` - возвращает `true` для пустого контейнера
- `size()` - возвращает количество элементов
- `capacity()` - возвращает вместимость контейнера
- `reserve(size_t count)` - выделяет память под `count` элементов
- `shrink_to_fit()` - уменьшает вместимость вектора в соответствии с `size()`

Модифицирующие методы:

- `assign()` - заменить текущее содержимое на новое
- `clear()` - очистить содержимое (без освобождения памяти)
- `push_back()` - добавить элемент в конец контейнера
- `pop_back()` - удалить последний элемент из контейнера
- `insert()` - добавить элементы в определенную позицию
- `erase()` - удалить элементы с определенной позиции
- `resize()` - изменить количество элементов контейнера
- `swap()` - обменять содержимое контейнеров.

Вместимость контейнера `std::vector`

По умолчанию, неинициализированный, пустой контейнер **НЕ** выделяет память под элементы.

Инициализированный контейнер, как правило, выделяет память ровно под количество элементов.

Методы, проверяющие вместимость и размер:

- `empty()` — возвращает true, если контейнер пуст
- `size()` — возвращает текущий размер, количество элементов в контейнере
- `capacity()` — возвращает вместимость контейнера, под какое количество элементов выделена память

Добавление в конец `std::vector`

Контейнер оптимизирован под добавление элементов в конец.

Метод, добавляющий элемент в конец:

- `push_back()` - добавить элемент, принимаемый в аргументах, в конец контейнера

Заполнение контейнера в цикле (не оптимально). Несколько раз в ходе заполнения контейнера произойдет реаллокация, то есть выделится новый блок памяти в куче, достаточный для элементов контейнера, и все элементы будут скопированы по новому адресу.

Как правило, при добавлении элемента в полностью заполненный контейнер, вместимость контейнера будет **увеличиваться в 2 раза** (libstdc++/libc++)

Сложность: **амортизированная O(1)**

```
std::vector<int> v;
size_t capacity = v.capacity();

std::cout << v.data() << std::endl; // int* (address)
std::cout << v.size() << ' ' << v.capacity() << std::endl;
for (int i = 0; i < 100; ++i) {
    v.push_back(i);
    if (capacity < v.capacity()) {
        capacity = v.capacity();

        std::cout << v.data() << std::endl; // int* (address)
        std::cout << v.size() << ' ' << v.capacity() << std::endl;
    }
}
```

Доступ к элементам `std::vector`

Доступ следует осуществлять только к валидным элементам.

Доступ к элементам осуществляется с помощью оператора `[]`, принимающего индекс, и с помощью методов:

- `[size_t pos]` — принимает индекс и возвращает ссылку на элемент
- `at(size_t pos)` — принимает индекс и возвращает ссылку на элемент, если переданный индекс (позиция) невалиден, то выбрасывается исключение `std::out_of_range`
- `front()` — возвращает ссылку на первый элемент
- `back()` — возвращает ссылку на последний элемент
- `data()` — возвращает сырой указатель на начало данных

Доступ к элементам вне диапазона контейнера, получение ссылок на элементы для пустого контейнера (`v.empty()` возвращает `true`) это **UB**

Итераторы

Итераторы - фундаментальная концепция стандартной библиотеки C++, выступающая в качестве обобщения указателей. Итераторы обеспечивают удобный **способ доступа** и **перемещения** по элементам различных контейнеров.

На данном этапе будем относиться к итераторам, как к удобной обертке над указателем.

Итераторы имитируют поведение **указателей**: разыменование, арифметику, сравнение

Существуют разные категории итераторов, поэтому в зависимости от контейнера поддерживаются не все операции с итераторами

Каждый контейнер имеет **свой тип итератора**.

Например, для вектора целочисленных значений тип итератора: `std::vector<int>::iterator`

Константная версия итератора имеет тип: `std::vector<int>::const_iterator`

Используются для реализации **range-based for** и в алгоритмах стандартной библиотеки.

Как правило, можно получить итератор на начало (первый элемент) **begin** и на конец **end** (за последний элемент). А также, их константные версии **cbegin** и **cend**

Итераторы `std::vector`

Итератор можно получить воспользовавшись методами контейнера:

- `begin()` — возвращает итератор на начало
- `end()` — возвращает итератор на конец
- `rbegin()` — возвращает обратный итератор на начало
- `rend()` — возвращает обратный итератор на конец

Также присутствуют методы с приставкой `c`, возвращающие их константные версии.

- `cbegin()` — возвращает константный итератор на начало
- `cend()` — возвращает константный итератор на конец
- `crbegin()` — возвращает константный обратный итератор на начало
- `crend()` — возвращает константный обратный итератор на конец

Чтобы не писать длинный тип итератора используется ключевое слово **auto**

Итерирование `std::vector`

Контейнер поддерживает различные способы итерирования, **range-based for**:

```cpp
std::vector<int> v = {1, 2, 3, 5};
for (auto vi : v) {
    std::cout << vi << std::endl;
}
```

Итерирование по индексу (порядковому номеру):

```cpp
std::vector<int> v = {1, 2, 3, 5};
for (size_t i = 0; i < v.size(); ++i) {
    std::cout << v[i] << std::endl;
}
```

Использование прямых итераторов:

```cpp
std::vector<int> v = {1, 2, 3, 5};
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << std::endl;
}
```

Использование обратных итераторов, для обхода контейнера в обратном порядке:

```cpp
std::vector<int> v = {1, 2, 3, 5};
for (auto it = v.rbegin(); it != v.rend(); ++it) {
    std::cout << *it << std::endl;
}
```

Метод `resize()`

Метод `resize()` изменяет размер контейнера, причем в случае увеличения создаются значения по умолчанию, в случае уменьшения вместимость контейнера, как правило, **НЕ** уменьшается

- `resize(size_t count)` — принимает требуемое количество элементов
- `resize(size_t count, const T& value)` — принимает требуемое количество элементов и значение элемента, которым необходимо заполнить контейнер в случае увеличения размера

Сложность: **O(n)** по отличию текущего размера от требуемого. **O(1)** при уменьшении для простых типов.

Тратятся ресурсы на заполнение **значениями по умолчанию**

Изменяет *size* контейнера, а также *capacity* при необходимости **увеличения**

Метод `reserve()`

Метод `reserve()` выделяет нужное количество памяти, не заполняя элементы значением по умолчанию, при этом *size* контейнера не изменяется, а *capacity* может только увеличиваться в соответствии с заданным значением

- `reserve(size_t new_capacity)` — принимает новое значение вместимости контейнера

Сложность: **O(n)** при реаллокации

Если известна требуемая вместимость заранее, наиболее оптимальный способ создания:

```
std::vector<int> v;
v.reserve(100);
for (size_t i = 0; i < 100; ++i) {
    v.push_back(i);
}
```

Метод `pop_back()`

Метод, удаляющий последний элемент:

- `pop_back()` — удаляет элемент из конца контейнера

Как правило, просто уменьшает размер контейнера, или перемещает указатель на конец элементов, **НЕ** вызывает реаллокацию памяти

Сложность: **O(1)**

`pop_back()` от пустого контейнера (`v.empty()` возвращает `true`) это **UB**

Метод `clear()`

Метод `clear()` очищает контейнер, *size* = 0, но *capacity* **НЕ** изменяется

Вызывает деструктор объектов. **НЕ** освобождает память

Сложность: **O(n)** при вызове деструкторов объектов, а для простых типов **O(1)**, как правило, просто переносится указатель конца элементов на начало

Остальные методы `std::vector`

Основные модифицирующие методы контейнера:

- `shrink_to_fit()` — уменьшает *capacity* в соответствии с текущим *size*, реаллокация, **O(n)**
- `insert()` — вставляет элемент или диапазон в произвольное место в контейнер, **O(n)**, вставка одного элемента в конец амортизированная **O(1)**. Возвращает итератор на начало вставки.

- `erase()` — удаляет элемент или диапазон из произвольного места контейнера, **O(n)**, поскольку необходимо передвинуть все элементы контейнера, удалить из конца **O(1)**. Возвращает итератор после последнего удаленного элемента.
- `swap()` — обменивает содержимое двух контейнеров, **O(1)** поскольку осуществляется обмен указателей
- `assign()` — заменяет содержимое контейнера на указанное значение или на диапазон, **O(n)**

и другие ...

Инвалидация итераторов `std::vector`

Более подробно данная тема будет затронута при обсуждении стандартной библиотеки STL и итераторов в частности.

Итератор, указатель или ссылка на элемент вектора указывают на конкретный адрес в динамической памяти. Некоторые методы перемещают элементы или освобождают память, после чего такие итераторы, указатели и ссылки становятся *невалидными* (аналогично висячей ссылке и невалидному указателю). Использование невалидного итератора, указателя или ссылки - **UB**.

Инвалидация итератора при вызове методов зависит от внутреннего устройства контейнера, поэтому для одноименных методов разных контейнеров итераторы могут вести себя по-разному.

Инвалидация итераторов `std::vector`:

- после реаллокации невалидны **все** итераторы, указатели и ссылки на элементы (`push_back()`, `insert()`, `reserve()`, `resize()` при росте *capacity*)
- `insert()` и `push_back()` без реаллокации инвалидируют итераторы начиная с позиции вставки и `end()`, итераторы до позиции вставки остаются валидными
- `erase()` и `pop_back()` инвалидируют итераторы начиная с позиции удаления
- `clear()` инвалидирует все итераторы

Без реаллокации старый итератор на практике указывает в ту же память, но элементы в ней уже сдвинуты, поэтому он ссылается на другой элемент. Стандарт считает такой итератор невалидным, и его разыменование - **UB**, даже если программа «работает»

```
std::vector<int> v = {1, 2, 3};    // capacity = 3
int* ptr = &v[0];
int& ref = v[1];
auto it = v.begin();

v.push_back(4);                    // reallocation
std::cout << *ptr << ref << *it;   // UB
```

- после реаллокации элементы скопированы по новому адресу, а старая память освобождена, санитайзер сообщает `heap-use-after-free`

```
std::vector<int> v = {1, 2, 3, 4, 5};
v.reserve(10);
auto first = v.begin();
auto third = v.begin() + 2;
v.erase(v.begin() + 1);   // without reallocation
std::cout << *first;      // OK, before the erase position
// std::cout << *third;   // UB, at or after the erase position
```

Методы `insert()` и `erase()` возвращают валидный итератор, поэтому удаление элементов в цикле следует писать так:

```
std::vector<int> v = {0, 1, 2, 3, 4, 5};
for (auto it = v.begin(); it != v.end();) {
    (*it % 2 == 0) ?
        it = v.erase(it) :  // iterator to the next element
        ++it;
}
// v = {1, 3, 5}
```

- если написать `v.erase(it); ++it;`, то инкрементируется уже невалидный итератор

Строка `std::string`

Строка `std::string` - контейнер стандартной библиотеки для хранения и обработки последовательности символов. В отличие от C-строки (массива `char` с завершающим `'\0'`), контейнер самостоятельно выделяет и освобождает память, а также хранит свой размер.

Заголовочный файл: `<string>`

Фактически `std::string` является псевдонимом `std::basic_string<char>`.

Символы строки, как и элементы вектора, располагаются в памяти непрерывно. Начиная с *C++11* гарантируется, что после последнего символа хранится `'\0'`, поэтому строку можно передать в C-функцию через метод `c_str()`.

Размер строки зависит от реализации. На 64-битной платформе это 32 или 24 байта.

Оптимизация коротких строк (*SSO*)

Small String Optimization (*SSO*) - оптимизация, при которой короткая строка хранит символы во внутреннем буфере самого объекта и не выделяет динамическую память.

Стандарт **НЕ** требует *SSO*, но все основные реализации её используют, а размер буфера зависит от реализации (64-битная платформа):

- **libstdc++** (GCC, Clang): до 15 символов, `sizeof(std::string)` равен 32
- **libc++** (Clang): до 22 символов, `sizeof(std::string)` равен 24
- **MSVC**: до 15 символов, `sizeof(std::string)` равен 32

```
std::string s;
std::cout << sizeof(s) << ' ' << s.capacity() << std::endl;  // 32 15
(libstdc++)
```

На Linux компилятор Clang берет по умолчанию библиотеку **libstdc++** от GCC.

Поэтому, если у вас также установлена **libc++** и хочется увидеть разный результат на своей машине, можно скомпилировать пример выше с соответствующим флагом:

```
clang++ -stdlib=libc++ main.cpp && ./a.out
```

Буфер находится внутри объекта, а **НЕ** обязательно на стеке: если сам объект строки лежит в куче (например, является элементом вектора), то и символы в куче.

Упрощенно, строку с *SSO* можно реализовать с помощью объединения `union` (близко к **libstdc++**):

```
class String {
    char* ptr_;     // points to local_buf_ or to heap memory
    size_t size_;
    union {
        char local_buf_[16];  // 15 chars + '\0'
        size_t capacity_;     // active when data in heap
    };
};
```

- если `ptr_` указывает на `local_buf_`, то строка короткая и активен буфер, иначе активна вместимость `capacity_`
- чтобы узнать режим, достаточно сравнить указатели, неактивный член объединения не читается

Также, можно реализовать строку следующим образом:

```
class String {
    size_t capacity_;
    union {
        struct {
            char* ptr;
            size_t size;
        } heap_buffer_;
        char stack_buffer_[sizeof(heap_buffer_)];
    };
};
```

- в зависимости от размера `capacity` обращение происходит к `stack_buffer_` или `heap_buffer_`
- названия `stack_buffer_` и `heap_buffer_` условны, говорят лишь о том, рядом с объектом строки находятся данные или в другом месте, поскольку положив строку в `std::vector`, она в любом случае будет храниться в куче

Следствие *SSO*: у короткой строки символы находятся внутри объекта, поэтому указатели и итераторы на символы строки могут стать невалидными после `swap()`, в отличие от вектора.

Создание строки

Существует множество способов создания строки.

```
std::string s1;                         // empty ""
std::string s2 = "Hello";               // from C-string literal
std::string s3("Hello", 4);             // "Hell", first 4 chars
std::string s4(5, 'a');                 // "aaaaa"
std::string s5{5, 'a'};                 // "\x05a", 2 chars!
std::string s6(s2, 1, 3);               // "ell", pos = 1, count = 3
std::string s7 = s2;                    // copy
std::string s8(s2.begin(), s2.end());   // iterators
```

- для `s5` фигурные скобки выбирают конструктор от списка инициализации, получается строка из двух символов с кодами `5` и `'a'`. Аналогично `std::vector<int> v{5, 2}` содержит два элемента, а `std::vector<int> v(5, 2)` пять. Подробнее в следующей лекции

Литерал `""s` (C++14)

Строковый литерал `"Hello"` имеет тип `const char[6]`, а не `std::string`. Для создания именно *std* строки есть встроенный литерал с суффиксом `s`.

Литерал расположен в пространстве имен: `std::literals::string_literals`

```
using namespace std::string_literals;

auto s1 = "Hello";   // const char*
auto s2 = "Hello"s;  // std::string

std::cout << std::string("a\0b").size() << std::endl;  // 1
std::cout << "a\0b"s.size() << std::endl;              // 3
```

- литерал удобен вместе с `auto` и когда строка содержит `'\0'`: конструктор от `const char*` останавливается на первом `'\0'`, а литерал знает свою длину
- для обычной инициализации `std::string s = "Hello";` литерал не нужен

Общие с `std::vector` методы

Строка поддерживает большинство методов вектора с той же сложностью:

- `size()`, `empty()`, `capacity()`, `reserve()`, `shrink_to_fit()`
- `[]`, `at()`, `front()`, `back()`, `data()`
- `push_back()`, `pop_back()`, `clear()`, `resize()`, `insert()`, `erase()`, `swap()`
- `begin()`, `end()`, `rbegin()`, `rend()` и их константные версии

Отличия от вектора:

- `length()` - синоним `size()`
- `c_str()` - возвращает `const char*` на строку, завершающуюся `'\0'`
- у пустой строки `data()` и `c_str()` возвращают валидный указатель на `'\0'`
- `s[s.size()]` возвращает ссылку на `'\0'`, изменение этого символа на другое значение - **UB**

Все методы можно посмотреть в справке

Конкатенация и присваивающие операторы

Для добавления в конец строки используются оператор `+=` и метод `append()`. Они принимают строку, C-строку или символ.

```
std::string str = "Hello";
str += ", my";
str += ' ';
str += std::string("friend");
str.append("!");
// str = "Hello, my friend!"
```

Оператор `+` создает новую строку. Хотя бы один из операндов должен быть `std::string`:

```
std::string str = "Hello" + std::string(", my ") + "friend!";
// std::string bad = "Hello" + ", my friend!";  // compile error
```

- в закомментированной строке складываются два массива `char`, для которых оператор `+` не определен

Начиная с *C++11* оператор `+` переиспользует память временной строки, поэтому цепочка `+` в одном выражении должна обходится недорого. Проблема возникает при накоплении строки в цикле:

```
std::string result;
for (int i = 0; i < 1000; ++i) {
    result = result + "ab";  // copy of whole result, O(n^2) in total
```

```
}

std::string better;
for (int i = 0; i < 1000; ++i) {
    better += "ab";          // amortized O(1), O(n) in total
}
```

- `result + "ab"` каждый раз создает новую строку, копируя все накопленные символы
- `+=` дописывает символы в конец, как `push_back()` у вектора, поэтому для накопления строки следует использовать присваивающие операторы

Изменение строки

Позиции и количество символов задаются индексами:

```
std::string s = "Hello world";
s.insert(5, ",");                   // "Hello, world"
s.erase(0, 7);                      // "world", pos = 0, count = 7
s.replace(0, 1, "W");               // "World"
std::string sub = s.substr(1, 3);   // "orl"
s.resize(3);                        // "Wor"
s.resize(5, '!');                   // "Wor!!"
```

- `substr(pos, count)` возвращает новую строку (копию части)
- если `count` выходит за конец строки, берутся символы до конца
- если `pos > size()`, выбрасывается исключение `std::out_of_range`

Поиск в строке

Методы поиска возвращают индекс первого символа найденной подстроки или специальное значение `std::string::npos`, если ничего не найдено.

- `find(str, pos = 0)` - поиск подстроки или символа, начиная с позиции `pos`
- `rfind(str)` - поиск с конца строки
- `find_first_of(chars)` - поиск первого символа из набора
- `find_first_not_of(chars)` - поиск первого символа не из набора
- `starts_with()`, `ends_with()` (*C++20*) - проверка начала и конца строки
- `contains()` (*C++23*) - проверка наличия подстроки

```
std::string s = "one two one";
size_t pos = s.find("one");         // 0
pos = s.find("one", 1);             // 8
pos = s.rfind("one");               // 8
pos = s.find_first_of("tw");        // 4
pos = s.find_first_not_of("one ");  // 4
```

```cpp
if (s.find('x') == std::string::npos) {
    std::cout << "not found" << std::endl;
}
s.starts_with("one");  // true
s.ends_with('e');      // true
```

- `npos` равно максимальному значению типа `size_t`, поэтому результат поиска следует сравнивать с `npos`, а **НЕ** проверять на отрицательность или сравнивать с `size()`

Сравнение строк

Для строк определены операторы сравнения, сравнение производится лексикографически. Вторым операндом может быть С-строка.

Метод `compare()` возвращает отрицательное число, `0` или положительное число.

```cpp
std::string a = "apple";
a == "apple";        // true
a < "banana";        // true
a.compare("apple");  // 0

const char* p = "apple";
const char* q = "apple";
p == q;              // compares addresses, NOT strings
```

- если оба операнда `const char*`, то сравниваются указатели, а **НЕ** символы.

Преобразование чисел и строк (*C++11*)

В стандартной библиотеке есть специальные функции для преобразования чисел и строк:

- `std::to_string(value)` - преобразует число в строку
- `std::stoi()`, `std::stol()`, `std::stoll()` - преобразует строку в целое число
- `std::stof()`, `std::stod()` - преобразует строку в число с плавающей точкой

```cpp
std::string s = std::to_string(42);    // "42"
std::string d = std::to_string(3.5);   // "3.500000"
int i = std::stoi("123");              // 123
double x = std::stod("2.5");           // 2.5

size_t idx = 0;
int j = std::stoi("42abc", &idx);      // 42, idx = 2
// std::stoi("abc");                   // exception std::invalid_argument
// std::stoi("99999999999");           // exception std::out_of_range
```

- `std::to_string` для чисел с плавающей точкой выводит 6 знаков после точки
- вторым аргументом `std::stoi` можно получить индекс первого непреобразованного символа

Ввод строки

Оператор `>>` считывает символы до первого пробельного символа. Для считывания строки целиком используется функция `std::getline`:

```
std::string word;
std::cin >> word;               // "Hello" from "Hello world"

std::string line;
std::getline(std::cin, line);   // whole line up to '\n'
```

После `std::cin >> value` во входном потоке остается символ `'\n'`, и следующий `std::getline` вернет пустую строку. Пропустить пробельные символы можно манипулятором `std::ws`:

```
int n;
std::cin >> n;
std::getline(std::cin >> std::ws, line);
```